React

Insieme a HTML e Javascript completa la parte di Front-End. React è un Framework (libreria Javascript) sviluppata da Meta(Facebook). Serve a costruire delle interfacce utente moderne ed interattive. con questa. libreria flessibile e potente si creeranno delle userinterface iterattive. é dichiarativa cioè si descriive cosa si vuole vedere sullo schermo e React si preoccupa di aggiornare il DOM in modo efficiente. Facilita le operazioni di aggiornamento di una pagina Web. Perchè usare React? Dispone di componenti riutilizzabili: ogni pezzo dell'interfaccia è un blocco indipendente, mantiene un codice ordinato e lo rende manutentabile.Permette aggiornamenti automatici significa. che quando cambia lo stato React aggiorna solo ciò che serve. Ha un enorme ecosistema di routing, gestione dati, libreria UI già pronte. React è usata ovunque per es. Netflix, Instagram, Airbnb.

Vediamo i concetti fondamentali:

1. ## Componenti

Sono funzioni che restituiscono JSX(una sintassi simile all'HTML).

Per es.

function Saluto(){ return

Ciao, mondo

}

Le espressioni JS si mettono tra {} In JSX si usa className e non class.

2. ## JSX

Permette di scrivere HTML dentro Javascript. In realtà viene tradotto in funzioni React.

3. ## Props

(sono gli input è immutabili nel componente figlio)

Sono degli argomenti che passiamo ai componenti: Per es.

function Saluto(){ return

Ciao, mondo

}

vediamo di defninire quale argomento da passare alla funzione Saluto:

function Saluto({nome}){ return

Ciao, {nome}

; }

questa funzione prende il nome passato come argomento e lo restituirà come risullato della funzione

4. State (useState)

Sono delle variabili React che ricorda tra un render ed un al'ltro, quando cambia l'interfaccia grafica si aggiorna in automatico. Per es:

import {useState} from "react";

function Contatore() { const [count,setCount] =useState(0); return (<button onClick = { () => setCount(count + 1)}>); Hai cliccato {count} volte! }

I nomi delle funzioni iniziano sempre con la lettera maiuscola.

5. Eventi

Si gestiscono come attributi JSX (onClick, onChange, etc) che accettano funzioni.

<button onClick = { () => alert(Cliccato)}>); Cliccami }

6. Liste e Chiavi

Servono per indirizzare array di elementi e serve una key unica:

definiamo un array:

const items = ["HTML", "CSS", "JS"];

- {items,map(item,i) =>
  {item}
  }

7. Eventi(useEffect)

Servono per svolgere "cose esterne" per es. modificare il titolo della pagina, timer, fetch API.

Per es:

import {useEffect} from "react"; useEffect(() => { document.title = "Pagina caricata"; }; []);

import {useEffect} from "react"; Importa l’hook useEffect da React. Gli hook sono funzioni speciali che ti permettono di usare funzionalità avanzate (come stato o effetti collaterali) nei componenti funzionali.

useEffect(() => { document.title = "Pagina caricata"; }, []);

useEffect prende due argomenti: una funzione di effetto (quello che deve succedere), un array di dipendenze (quando deve succedere). In questo caso: La funzione cambia il titolo della pagina in "Pagina

caricata" modificando document.title. L'array [] vuoto significa che l'effetto verrà eseguito solo una volta, al "montaggio" del componente (simile a componentDidMount nelle classi React).

Quando questo frammento viene eseguito dentro un componente React, appena il componente viene caricato nel DOM: il titolo della scheda del browser diventa "Pagina caricata

Come funziona React??

Mantiene una sorta di rapprentazione interna della user-interface chiamata Virtual DOM. Quando lo stato cambia allora React calcola le differenza e aggiorna solo le parti necessarie del vero DOM in modo da avere un output aggiornato in maniera veloce.

Come si usa?

1. Prepariamo un progetto con Vite o Next.js
2. Scriviamo. componenti che descrivono la nostra UI.
3. Gestiamo stato ed eventi per renderla interattiva.
4. Usiamo librerie esterne per routing, gesitone dati, animazioni, etc.
5. Compiliamo in produzione (npm run build) e pubblichiamo online.

Esercizio n°1

Crea un componente chiamato titolo che riceve un testo e lo mostra dentro un h1 con una classsName = "title".

function Titolo ({testo}) { return

{testo}

; }

Cosa installare

Aprire terminale Controllare se è già installato:

- node -v
- npm -v

https://nodejs.org scaricare la versione LTS (Long Term SUpport) installare e riavviare terminale

- git --version oppure git -v

Per creare un progetto in React:

Scegliere una cartella di lavoro e da terminale digitare:

- npm create vite@latest my-react-app

Durante la procedura scegli: React come framework JavaScript o TypeScript (a tua scelta)

Entrare nella cartella creata:

- cd my-react-app (si trova dentro la cartella React)

Installare le dipendenze:

- npm install

Avvia il server di sviluppo:

- npm run dev

L'output:

VITE v7.1.5 ready in 500 ms

➜ Local: http://localhost:5173/

Aprire il link http://localhost:5173/ nel browser in modo tale da aprire la pagina iniziale di React

Da lì si può aprire il progetto in Visual Studio Code e modificare i file dentro src/.

Struttura Progetto React

my-react-app/

- **index.html** #Pagina principale
- *src/*
- **App.jsx** #Primo componente React
- **main.jsx** #Monta React nell''HTML
- **index.css** #Stili globali

Modificare il primo componente

Aprire con VS in my-react-app src/App.jsx e cancellaret utto il contenuto all'interno e si inserisce il seguente:

export default function App() { return (

Ciao React

Questa è la mia prima paginadi React

); }

Aggiungere delle interattività: import { useState } from "react"; export default function App() { const [count, setCount] = useState(0); return (

Ciao React

Questa è la mia prima paginadi React. Hai cliccato {count} volte

setCount(count + 1)}>Cliccami
); }

Per stoppare il progetto premere control + c e si chiude il server, per riavviare si lancia il comando -> npm run dev

---------- Dal terminale andare nella cartella del progetto React: cd ~/Scrivania/my-react-app code . Questo aprirà direttamente la cartella my-react-app in VS Code

NB: ~ è equivalente a /Users/giordanapandolfo

- cd ~/Scrivania/my-react-app
- cd /Users/giordanapandolfo/Scrivania/my-react-app

---------- Nel progetto my-react-Base assets,components(Titolo.jsx),data(projeect.js) e styles(dentro c'è index.html) sono cartelle create da noi su src

---

Esercizio: Aggiungere un contatore dei progetti trovati Modificare il progetto my-react-base per mostrare, sopra la lista, quanti progetti corrispondono alla ricerca,si utiilizza l'operatore ternario.

Operatore ternario

{filtered.length} progetto{filtered.length !== 1 ? "i" : ""} trovati

condizione ? valoreSeVero : valoreSeFalso La parte {filtered.length !== 1 ? "i" : ""} è un ternario. Sintassi:

condizione ? valoreSeVero : valoreSeFalso

Se filtered.length !== 1 (cioè non è 1) → restituisci "i" Altrimenti → restituisci "" (stringa vuota)

---

0 ? "green":""}`} >

- className: Per assegnare classi CSS, serve a collegare il CSS all'elemento.
- `muted ${...}`: Questo è un **template literal** in JavaScript (stringa tra backtick `). Serve a combinare più classi dinamicamente. Permette interpolazione di variabili, stringhe su più righe,inserire espressioni complesse

filtered.length === 0 ? "red" Se non ci sono progetti, aggiunge la classe red → testo rosso.

: filtered.length > 0 ? "green":"" altrimenti, se c'è almeno un progetto, aggiunge la classe green → testo verde.